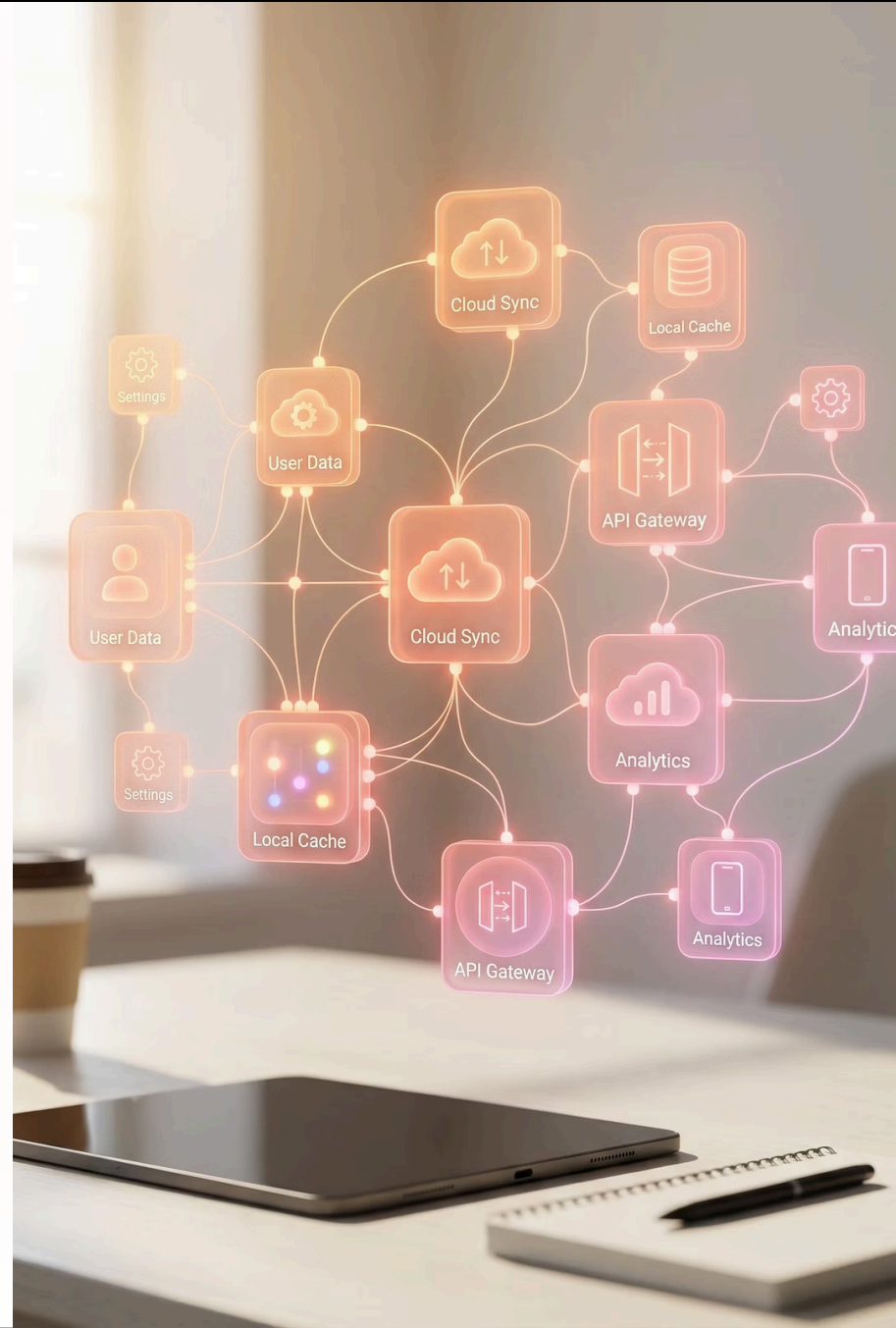
Room: SQLite Persistence for Android

By: Tal Zion

https://developer.android.com/training/data-storage/room#kts



Room: Your SQLite Wrapper

What is Room?

Room is a powerful persistence library that sits on top of SQLite, providing an abstraction layer that makes database operations more intuitive and type-safe. It's part of Android Jetpack and designed to work seamlessly with modern Android architecture components.

Why Use Room?

Room eliminates much of the boilerplate code required for SQLite database operations while maintaining compile-time verification of SQL queries. This reduces runtime errors and makes your code more maintainable and testable.

Room Persistent Library: Core Architecture

Room provides an abstraction layer over SQLite to allow fluent database access while harnessing the full power of SQLite. Understanding its three major components is essential for effective implementation.

Database

The database holder serves as the main access point to your app's persisted data. The class annotated with @Database must be an abstract class extending RoomDatabase, include the list of entities, and contain abstract methods returning DAO classes.

Entity

Represents a table within the database. Each entity class defines the schema for a database table, with fields mapping to columns. Entities use annotations to specify primary keys, column names, and relationships.

DAO

Data Access Object contains the methods used for accessing the database. DAOs define the database operations through annotated methods, providing a clean API for database interactions without writing SQL manually.

The class annotated with @Database should be an abstract class that extends RoomDatabase, include the list of entities associated with the database within the annotation, and contain an abstract method with 0 arguments returning the @Dao annotated class.

Creating Database Instances

Runtime Database Instantiation

At runtime, you acquire an instance of Database by calling Room.databaseBuilder() for persistent storage or Room.inMemoryDatabaseBuilder() for temporary in-memory databases. The builder pattern allows you to configure various database options before creation.

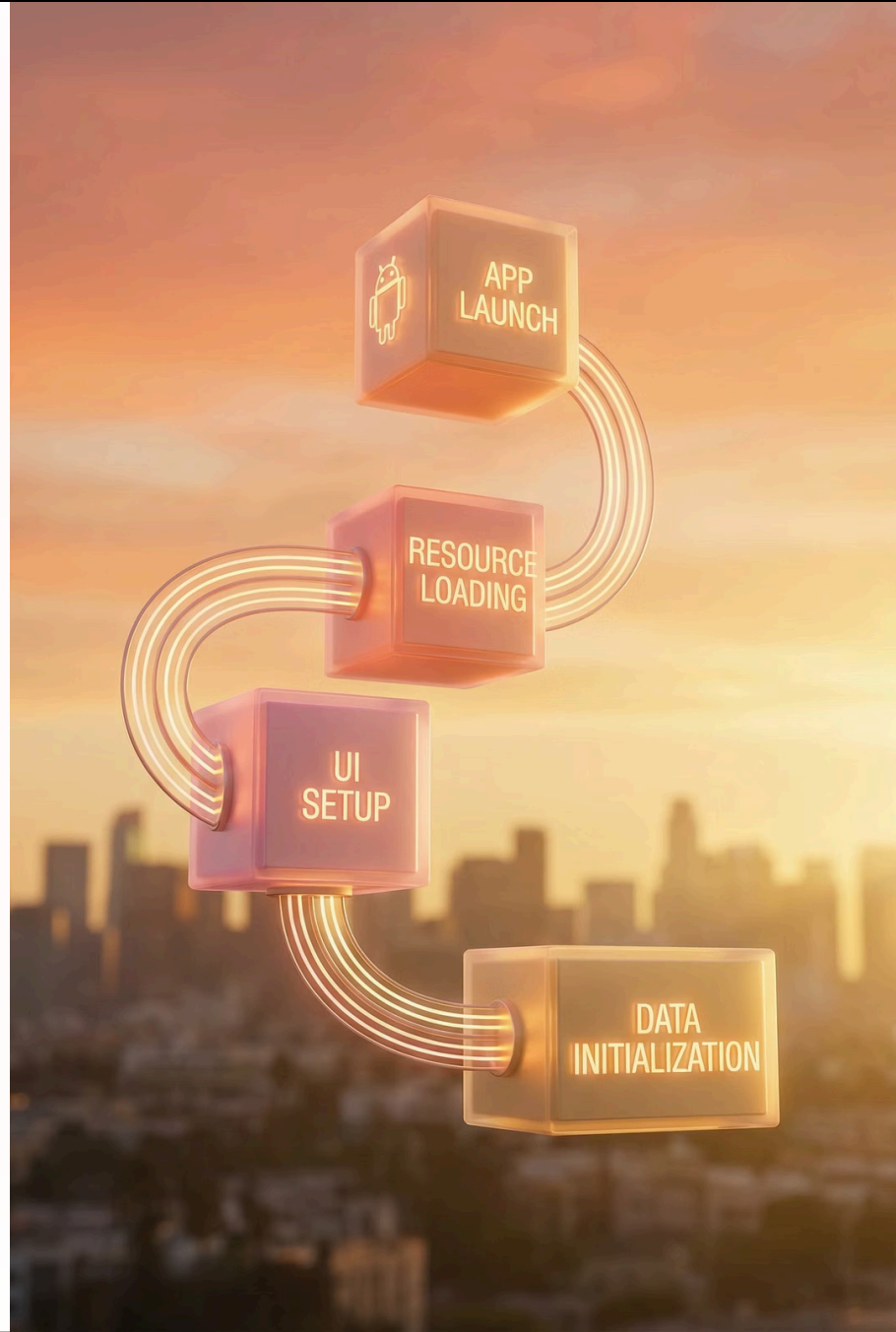
The Three Components Work Together

- **Entity:** Represents a table within the database, defining the data structure
- **DAO:** Contains the methods used for accessing the database with type-safe queries
- **Database:** Ties everything together and provides the main access point

Builder Methods

databaseBuilder(): Creates a persistent database saved to disk that survives app restarts.

inMemoryDatabaseBuilder(): Creates a temporary database stored in memory, useful for testing.



Setting Up Dependencies

Before implementing Room in your project, you need to add the required dependencies to your app-level build.gradle file. Room requires both runtime and compiler dependencies, plus Kotlin extensions for coroutine support.

Version Declaration

```
def room_version = "2.8.4"
```

Define the Room version at the top of your gradle file for easy maintenance and consistency across dependencies.

Core Dependencies

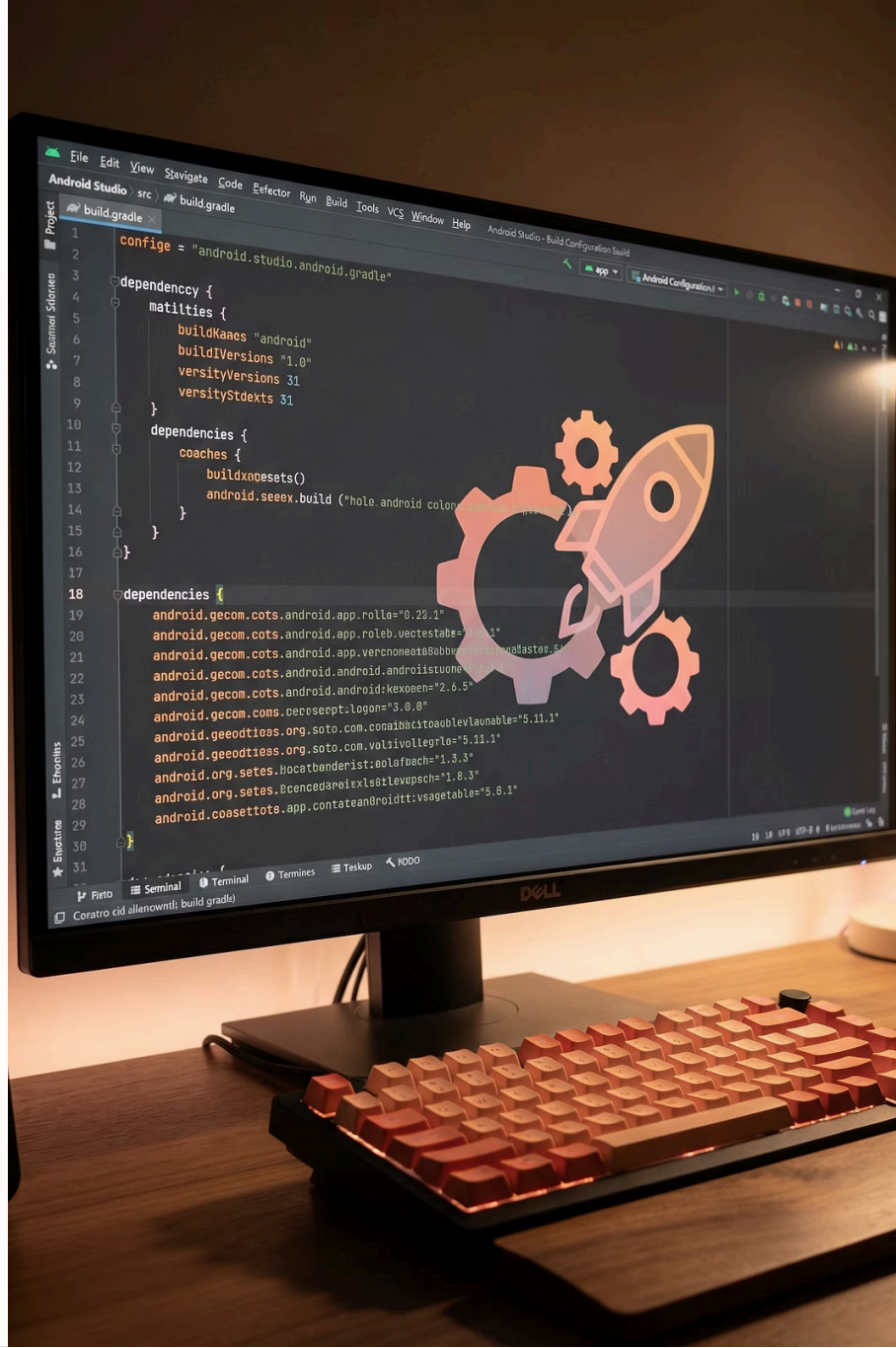
```
implementation "androidx.room:room-runtime:$room_version"
kapt "androidx.room:room-compiler:$room_version"
implementation "androidx.room:room-ktx:$room_version"
```

The runtime library provides the core Room functionality, the compiler generates implementation code, and ktx adds Kotlin extensions.

Plugin Configuration

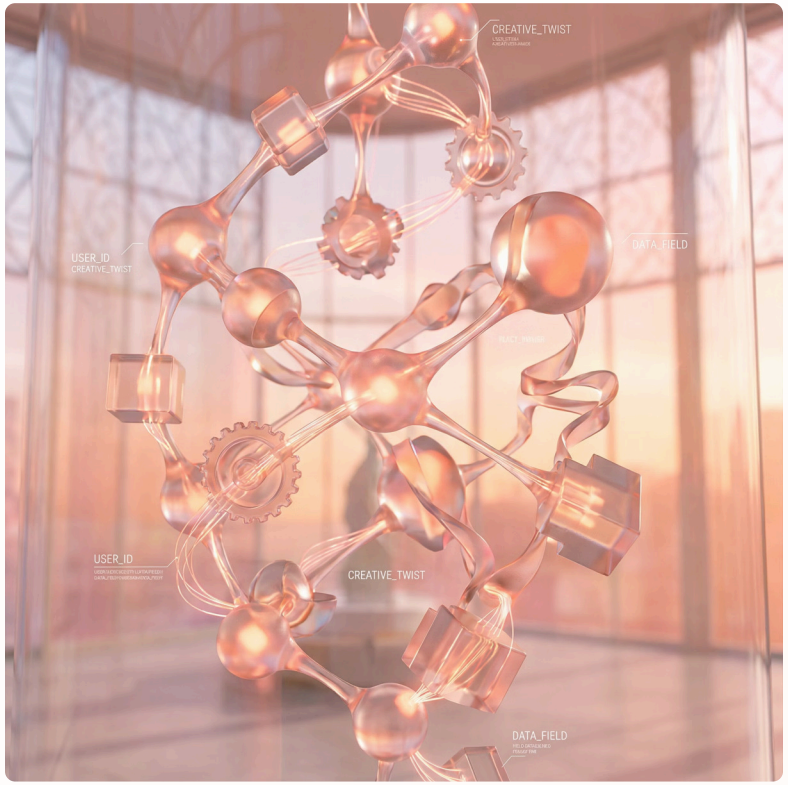
```
plugins {
    id 'kotlin-kapt'
}
```

The Kotlin annotation processing plugin (kapt) is required to process Room's annotations at compile time.



Defining Your First Entity

Entities are Kotlin data classes that represent tables in your database. Each instance of an entity represents a row in the table. The @Entity annotation marks a class as a database table, while @PrimaryKey designates the unique identifier.



Student Entity Example

```
@Entity
data class Student(
    @PrimaryKey val name: String,
    val id: String,
    val avatar: String,
    var isChecked: Boolean
)
```

This Student entity creates a table with four columns. The name field serves as the primary key, ensuring each student has a unique name. Properties automatically map to table columns unless specified otherwise.

Each property in the data class becomes a column in the database table. Room handles the type conversions automatically for common types like String, Int, and Boolean.

Creating Data Access Objects (DAOs)

DAOs define the methods for accessing your database. They provide a clean, type-safe API for database operations. The @Dao annotation marks an interface or abstract class as a Data Access Object, and method annotations like @Query, @Insert, and @Delete define the operations.

Query All Records

```
@Query("SELECT * FROM Student")
fun getAll(): List<Student>
```

Retrieves all students from the database table.

Query by ID

```
@Query("SELECT * FROM Student WHERE id =:id")
fun getStudentById(id: String): Student
```

Finds a specific student using their ID parameter.

Insert Operations

```
@Insert(onConflict = OnConflictStrategy.REPLACE)
fun insertAll(vararg students: Student)
```

Inserts one or more students, replacing on conflict.

Delete Operations

```
@Delete
fun delete(student: Student)
```

Removes a student record from the database.

Implementing the Database Class

The Database class ties together your entities and DAOs. It must be an abstract class annotated with @Database that extends RoomDatabase. This class serves as the main access point for database interactions in your app.

Database Declaration

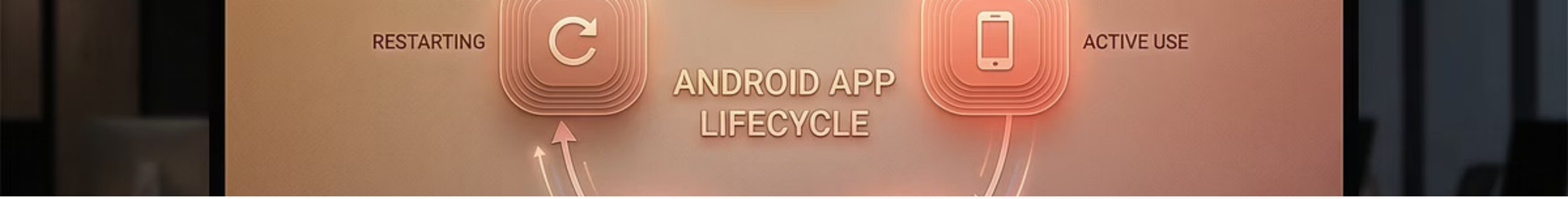
```
@Database(entities = [Student::class], version = 1)
abstract class AppLocalDbRepository : RoomDatabase() {
    abstract fun studentDao(): StudentDao
}
```

The @Database annotation specifies the entities and database version. The abstract DAO method provides access to database operations.

Singleton Pattern

```
object AppLocalDb {
    val db: AppLocalDbRepository by lazy {
        val context = MyApplication.Globals.appContext
            ?: throw IllegalStateException("Context not available")
        Room.databaseBuilder(
            context,
            AppLocalDbRepository::class.java,
            "dbFileName.db"
        ).fallbackToDestructiveMigration()
         .build()
    }
}
```

Using lazy initialization ensures the database is created only when first accessed, improving app startup performance.



Accessing Application Context

Room requires an application context to create the database instance. By extending the Application class, you can maintain a global reference to the context that's available throughout your app's lifecycle.

01

Declare in Manifest

In your AndroidManifest.xml file, specify your custom Application class:

```
<application
    android:name=".base.MyApplication"
```

02

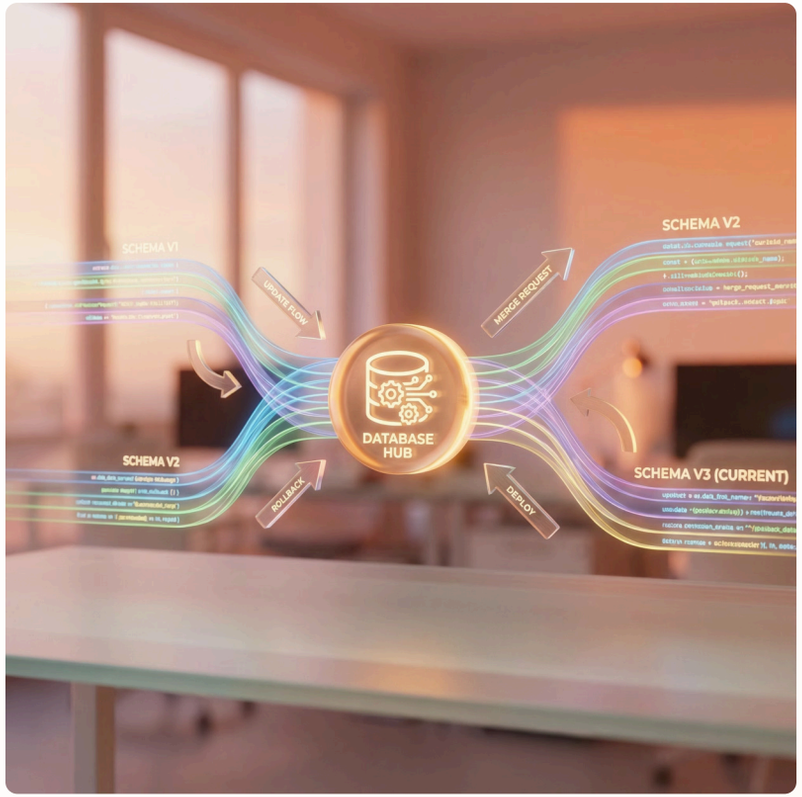
Create Application Class

Implement your custom Application class with a global context reference:

```
class MyApplication: Application() {
    object Globals {
        var appContext: Context? = null
    }
    override fun onCreate() {
        super.onCreate()
        Globals.appContext = applicationContext
    }
}
```

The Application class onCreate() method is called before any activity, service, or receiver objects are created, making it the perfect place to initialize global resources like the database context.

Database Schema Export



Why Export Schema?

Exporting your database schema allows you to track changes across different versions, debug migration issues, and maintain clear documentation of your database structure over time.

Gradle Configuration

Room can automatically generate schema files in JSON format that document your database structure. Add this configuration to your app-level build.gradle:

```
defaultConfig {
    javaCompileOptions {
        annotationProcessorOptions {
            arguments = ["room.schemaLocation":
                    "$projectDir/schemas".toString()]
        }
    }
}
```

This creates a schemas directory in your project with a JSON file for each database version, making it easier to track structural changes and debug migration problems.

Understanding the Generated Schema

Room generates a comprehensive JSON schema file that documents your database structure. This file is invaluable for debugging, migrations, and understanding your database evolution over time.

Schema Structure

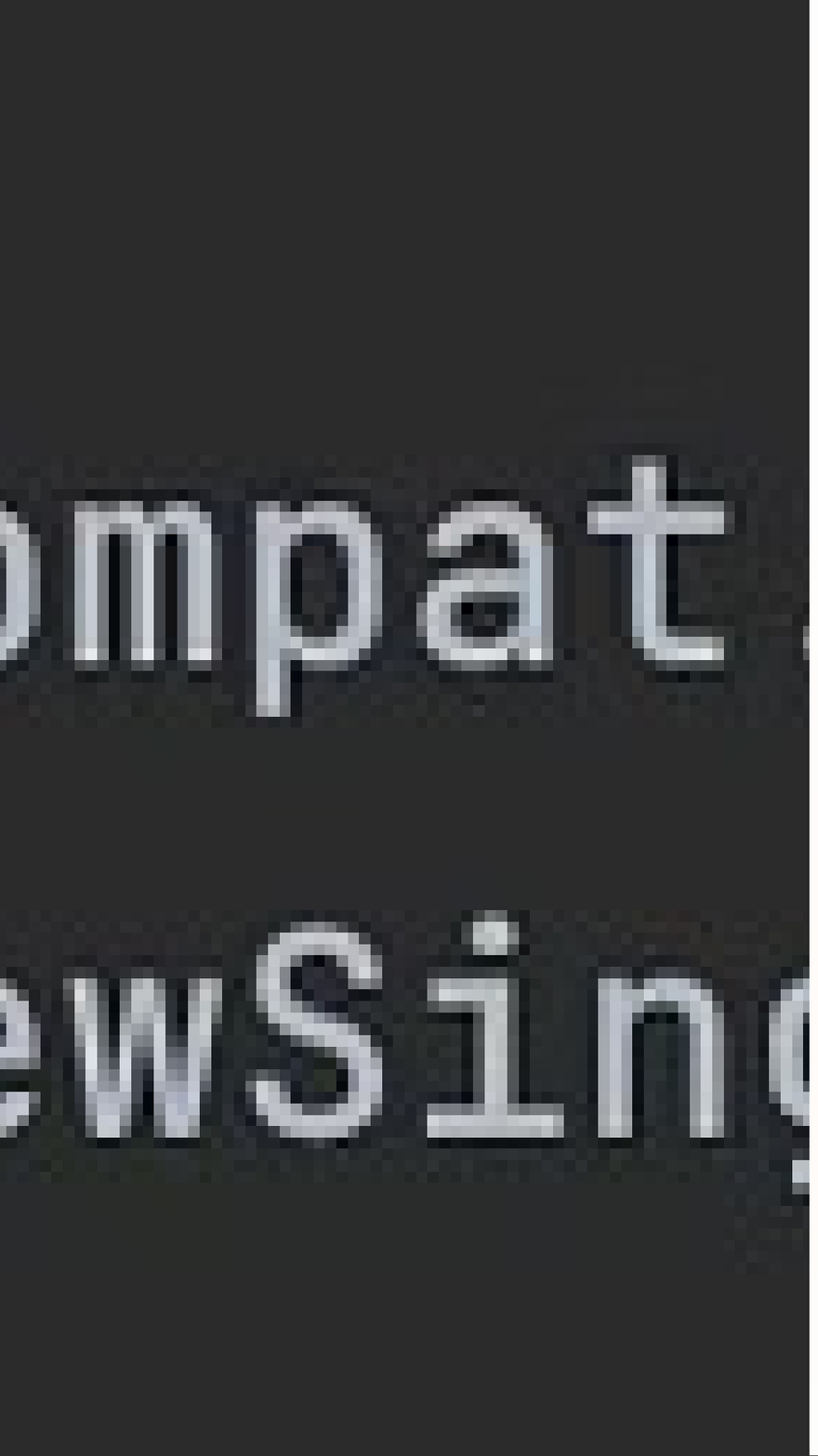
```
{
 "formatVersion": 1,
 "database": {
  "version": 6,
  "identityHash": "d0c839e76460c10cfb13733603b70ab1",
  "entities": [
   {
    "tableName": "Student",
    "createSql": "CREATE TABLE IF NOT EXISTS `${TABLE_NAME}`
            (`id` TEXT NOT NULL, `name` TEXT,
             `image` TEXT, PRIMARY KEY(`id`))",
    "fields": [
     {
      "fieldPath": "id",
      "columnName": "id",
      "affinity": "TEXT",
      "notNull": true
     }
```

Key Schema Elements

- **version:** Database version number for migration tracking
- **identityHash:** Unique hash identifying this schema structure
- **tableName:** The name of each table in the database
- **createSql:** The SQL statement used to create the table
- **fields:** Detailed information about each column including type, nullability, and constraints

The schema file serves as documentation and validation for database migrations, helping prevent data loss during app updates.

Making DAO Asynchronous



The Threading Challenge

By default, Room's DAO operations are synchronous, meaning they execute on the calling thread. Running database operations on Android's main UI thread causes the dreaded "Execute on Main Thread Exception" and leads to app freezes and poor user experience.

The Problem

Current DAO implementations are synchronous and will crash if executed on the main thread. Android requires all potentially long-running operations to run on background threads to keep the UI responsive.

The Solution

We need to make our DAO and Model layer asynchronous by executing database operations on background threads, then returning results to the main thread for UI updates.

```
fun getAllStudents(callback: (List<Student>) -> Unit) {
    executor.execute {
        val students = appDatabase.studentDao().getAll()
        callback(students)
    }
}
```

Background Threading in Android

Understanding Android's threading model is crucial for building responsive applications. The official Android documentation provides comprehensive guidance on background threading strategies.

> Reference: https://developer.android.com/guide/background/threading

Main Thread Responsibilities

The main thread (also called the UI thread) handles all user interface operations, event processing, and view drawing. It must remain responsive to user interactions, typically completing operations within 16ms to maintain 60fps.

Background Thread Usage

Background threads handle time-consuming operations like network requests, database queries, file I/O, and complex computations. This prevents UI freezes and maintains smooth user experience.

Android Background Tasks Architecture

All Android apps use a main thread to handle UI operations. Calling long-running operations from this main thread can lead to freezes and unresponsiveness, creating a poor user experience.

Main Thread

Handles all UI operations and must remain responsive. Any operation taking longer than a few milliseconds should move to a background thread.

Background Thread

Executes long-running operations like database queries, network calls, and file operations without blocking the UI.

Return to Main

After the background thread finishes, updating the UI must be performed by the main thread to prevent threading violations.

Thread Pools and Executors

A thread pool is a managed collection of threads that runs tasks in parallel from a queue. Creating threads is expensive, so you should create a thread pool only once as your app initializes. This pattern improves performance by reusing existing threads for new tasks.

ExecutorService Implementation

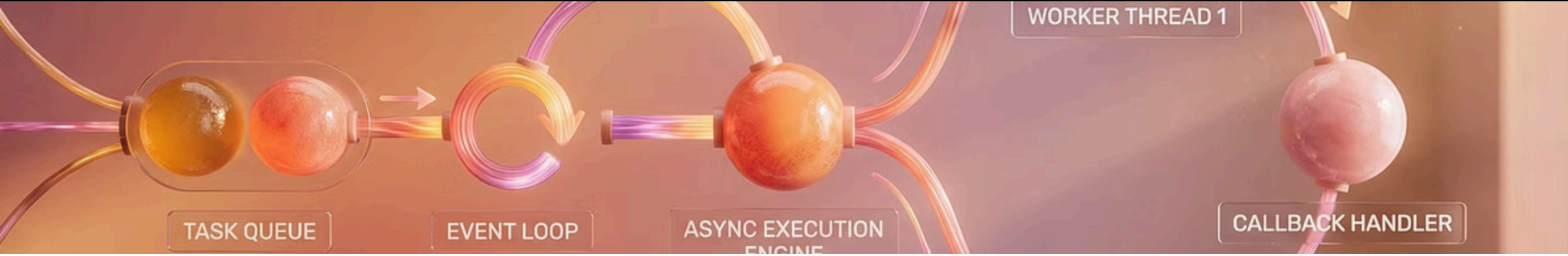
To send a task to a thread pool, use the ExecutorService interface. New tasks are executed on existing threads from the pool, avoiding the overhead of thread creation for each operation.

```
public class MyApplication extends Application {
    ExecutorService executorService =
        Executors.newFixedThreadPool(4);
}
```

This creates a fixed pool of 4 worker threads that will handle all background database operations. The number of threads should be based on your app's needs and device capabilities.

Benefits of Thread Pools

- Reuses threads to reduce overhead
- Limits concurrent threads to prevent resource exhaustion
- Manages task queuing automatically
- Improves app performance and responsiveness



Executing Background Operations

Once you've configured your ExecutorService, you can submit database operations to run on background threads. The executor handles thread management while your code focuses on the business logic.

Background Operation Pattern

Submit your database operations to the executor service using the execute() or submit() method. The operation runs on a background thread from the pool, keeping the main thread free for UI updates.

Implementation Example

```
Model.instance.getAllStudents { students ->
    this.students = students
}
```

The executor manages the background thread lifecycle, allowing you to focus on the database operation itself rather than thread management details.

Returning Results to Main Thread

After completing background operations, you must return to the main thread to update the UI. Android provides the Handler class for posting operations back to the main thread safely.

01

Create Main Thread Handler

Initialize a Handler tied to the main thread's Looper, typically in your Application class for global access:

```
private var mainHandler = HandlerCompat.createAsync(Looper.getMainLooper())
```

02

Post to Main Thread

Use the handler to post your UI update callback to the main thread after the background operation completes:

```
fun getAllStudents(callback: (List<Student>)
    executor.execute {
        val students = appDatabase.studentDa
        mainHandler.post {
            callback(students)
```

The Handler ensures that UI updates only happen on the main thread, preventing threading violations and potential crashes. This pattern is essential for safe asynchronous database operations.

Advanced Query Examples

Room supports complex SQL queries through its @Query annotation. These examples demonstrate various query patterns you can use to retrieve and manipulate data efficiently.

```
@Dao
interface StudentDao {

    @Query("SELECT * FROM Student")
    fun getAllStudents(): List<Student>

    @Query("SELECT * FROM Student WHERE id =:id")
    fun getStudentById(id: String): Student

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    fun insertStudents(vararg students: Student)

    @Delete
    fun delete(student: Student)
}
```

Query Capabilities

- Parameterized queries with type safety
- Complex WHERE clauses with multiple conditions
- JOIN operations across multiple tables
- Aggregate functions like COUNT, SUM, AVG
- ORDER BY and GROUP BY clauses

Best Practices

- Use parameter binding to prevent SQL injection
- Leverage Room's compile-time query verification
- Return LiveData or Flow for reactive updates
- Keep queries focused and efficient
- Index frequently queried columns

Displaying Progress Indicators

While fetching data from the database, you need to display visual feedback that work is in progress. This prevents user confusion and provides a better user experience during asynchronous operations.

Show Progress Indicator

Display a progress bar or loading spinner before starting the database operation. This immediately provides visual feedback that the app is working on the user's request.

Execute Background Task

Run your database query on a background thread using the ExecutorService. The progress indicator remains visible while the operation executes, keeping the UI responsive.

Hide on Completion

Once the background operation completes and returns to the main thread, hide the progress indicator and update the UI with the retrieved data.

Common progress indicators include ProgressBar, circular loading animations, skeleton screens, or shimmer effects. Choose the indicator that best fits your app's design language and the expected operation duration.